

MTasking

Sistema de Gestion de Tareas

Politica de mantenimiento de software

1. Introduccion

MTasking es un sistema web de gestion de tareas desarrollado como proyecto academico para la materia Construccion de Software. Esta implementado con PHP sin frameworks, Vanilla JS y MySQL, desplegado localmente sobre XAMPP. Esta politica establece los lineamientos, procedimientos y responsabilidades para garantizar la estabilidad, calidad y evolucion controlada del sistema a lo largo de su ciclo de vida.

2. Objetivo

Definir el marco de trabajo para el mantenimiento del sistema MTasking, asegurando que cualquier modificacion, correccion o mejora se realice de manera ordenada, trazable y sin comprometer la integridad de las funcionalidades existentes.

3. Alcance

Esta politica aplica a todos los componentes del sistema:

- Backend PHP: controladores, modelos, rutas, excepciones y servicios
- Frontend: paginas HTML, hojas de estilo CSS y logica JavaScript
- Base de datos MySQL: esquema, migraciones y datos de prueba
- Infraestructura de tests: PHPUnit con SQLite en memoria
- Configuracion del entorno: XAMPP, Apache puerto 8083

4. Tipos de Mantenimiento

4.1 Correctivo

Correccion de defectos detectados durante pruebas o en el entorno de ejecucion. Incluye errores fatales de PHP, fallos de logica en controladores y problemas de integridad en la base de datos. Tiene prioridad maxima y debe atenderse antes de cualquier nuevo desarrollo.

4.2 Preventivo

Acciones para reducir la probabilidad de fallos futuros: refactoring de codigo duplicado, actualizacion de dependencias como PHPMailer y PHPUnit, revision periodica de la jerarquia de excepciones y limpieza automatica de tokens expirados en la tabla password_resets.

4.3 Perfectivo

Mejoras a funcionalidades existentes: optimizacion de consultas SQL, mejora de mensajes de error en el frontend, ajustes de experiencia de usuario en modales de tareas y comentarios, y aumento de cobertura de tests unitarios.

4.4 Adaptativo

Cambios requeridos por variaciones del entorno: migracion a nuevas versiones de PHP, cambios en la configuracion SMTP del servidor de correo, ajuste del puerto de Apache, o modificaciones al esquema SQL ante nuevos requerimientos academicos del segundo parcial.

5. Procedimiento de Gestion de Cambios

1. Identificacion

El integrante detecta el defecto o necesidad de mejora y abre un issue en el repositorio GitHub, describiendo el problema, el componente afectado y el impacto esperado.

2. Analisis y aprobacion

El responsable de revision de codigo evalua la solicitud. Los cambios correctivos urgentes se aprueban directamente; los cambios perfectivos o adaptativos requieren consenso del equipo.

3. Desarrollo en rama

Se crea una rama con la convencion feature/ o fix/ a partir de main. Queda prohibido desarrollar directamente sobre la rama principal.

4. Revision de codigo (PR)

Se abre un Pull Request en GitHub. El revisor verifica correctitud funcional, adherencia a la arquitectura, compatibilidad con los tests y ausencia de credenciales en el codigo.

5. Pruebas

Se ejecuta la suite PHPUnit (php phpunit.phar) y se verifica manualmente el flujo afectado en XAMPP. El PR no puede mergearse si algun test existente falla.

6. Merge y registro

Aprobado el PR, se realiza el merge a main. El integrante registra el cambio en su bitacora individual indicando: fecha, tipo de mantenimiento, archivos modificados y resultado obtenido.

6. Reglas Tecnicas de Mantenimiento

Stack fijo: No se incorporaran nuevos frameworks, ORMs ni librerias de terceros sin aprobacion del equipo. Excepcion documentada: PHPMailer para el modulo de recuperacion de contrasena.

Sin credenciales en el repositorio: Contraseñas, tokens SMTP y claves de API deben mantenerse en archivos de configuracion excluidos del repositorio via .gitignore, distribuidos como archivos *.example.php.

Compatibilidad con tests: Todo cambio en modelos o controladores debe mantener los tests PHPUnit existentes en verde. Los nuevos modulos deben incluir tests usando SQLite en memoria.

Esquema de base de datos: Cualquier adicion o modificacion de tablas debe reflejarse en database/schema.sql con sentencias CREATE TABLE IF NOT EXISTS o ALTER TABLE.

Jerarquia de excepciones: Los errores de negocio deben lanzarse mediante ApplicationException o sus subclases. Queda prohibido el uso de die() o http_response_code() sin respuesta JSON asociada.

Convencion de respuestas API: Todos los endpoints retornan JSON. Codigos HTTP: 200 exito, 201 creacion, 400 datos invalidos, 401 no autenticado, 404 no encontrado, 500 error de servidor.